

AFRILANG Stdlib

std/c/tscumulative

Français. Bibliothèque AFRILANG 'std/c/tscumulative' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : cumuSum, cumuMean, cumuMin, cumuMax, cumuVar, cumuStd, cumuNorm.

```
'import "std/c/tscumulative"' · 'use tscumulative'
```

```
- 'cumuSum(v list number) → number' - 'cumuMean(v list number) →  
number' - 'cumuMin(v list number) → number' - 'cumuMax(v list num-  
ber) → number' - 'cumuVar(v list number) → number' - 'cumuStd(v list  
number) → number' - 'cumuNorm(v list number) → list number'
```

English. AFRILANG library 'std/c/tscumulative' (tier complex, category complex). Exports 7 function(s): cumuSum, cumuMean, cumuMin, cumuMax, cumuVar, cumuStd, cumuNorm.

```
'import "std/c/tscumulative"' · 'use tscumulative'
```

```
- 'cumuSum(v list number) → number' - 'cumuMean(v list number) →  
number' - 'cumuMin(v list number) → number' - 'cumuMax(v list num-  
ber) → number' - 'cumuVar(v list number) → number' - 'cumuStd(v list  
number) → number' - 'cumuNorm(v list number) → list number'
```

Catégorie / Category	Modules complex (c/)	
Niveau / Tier	complex	June 16, 2026
Fonctions / Functions	7	

Contents

Français

1. Vue d'ensemble

Bibliothèque AFRILANG 'std/c/tscumulative' (niveau complex, catégorie complex). Elle expose 7 fonction(s) : cumuSum, cumuMean, cumuMin, cumuMax, cumuVar, cumuStd, cumuNorm.

```
'import "std/c/tscumulative"' · 'use tscumulative'
```

```
- `cumuSum(v list number) → number`
- `cumuMean(v list number) → number`
- `cumuMin(v list number) → number`
- `cumuMax(v list number) → number`
- `cumuVar(v list number) → number`
- `cumuStd(v list number) → number`
- `cumuNorm(v list number) → list number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/tscumulative"
use tscumulative
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés – graphes, NLP, réseaux complexes.

3. Référence API

- cumuSum(v list number) → number•
cumuMean(v list number) → number•
cumuMin(v list number) → number•
cumuMax(v list number) → number•
cumuVar(v list number) → number•
cumuStd(v list number) → number•
cumuNorm(v list number) → list

4. Exemples d'utilisation

```
import "std/c/tscumulative"
use tscumulative
```

```
create result = cumuSum(/* args */)
say result
```

```
create result = cumuMean(/* args */)
say result
```

```
create result = cumuMin(/* args */)
say result
```

```
create result = cumuMax(/* args */)
say result
```

```
create result = cumuVar(/* args */)
say result
```

```
create result = cumuStd(/* args */)
say result
```

5. Bonnes pratiques

- Préférez ``import "std/c/tscumulative"`` en tête de fichier.
- Lancez ``afriLang check`` avant de livrer.
- Combinez avec les modules `stdlib` de la même catégorie.
- Voir `/stdlib/` pour le source live et les démos playground.
- Signalez les problèmes sur GitHub si une export manque.

6. Annexe — source

module tscumulative

```
function cumuSum(v list number) returns number
end
```

```
function cumuMean(v list number) returns number
end
```

```
function cumuMin(v list number) returns number
end
```

```
function cumuMax(v list number) returns number
end
```

```
function cumuVar(v list number) returns number
end
```

```
function cumuStd(v list number) returns number
end
```

```
function cumuNorm(v list number) returns list number
end
end
```

English

1. Overview

AFRILANG library 'std/c/tscumulative' (tier complex, category complex). Exports 7 function(s): cumuSum, cumuMean, cumuMin, cumuMax, cumuVar, cumuStd, cumuNorm.

'import "std/c/tscumulative"' · 'use tscumulative'

```
- `cumuSum(v list number) → number`
- `cumuMean(v list number) → number`
- `cumuMin(v list number) → number`
- `cumuMax(v list number) → number`
- `cumuVar(v list number) → number`
- `cumuStd(v list number) → number`
- `cumuNorm(v list number) → list number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/tscumulative"
use tscumulative
```

Tier: complex · Category: Complex modules (c/)
Advanced domains – graphs, NLP, complex networks.

3. API reference

- cumuSum(v list number) → number•
cumuMean(v list number) → number•
cumuMin(v list number) → number•
cumuMax(v list number) → number•
cumuVar(v list number) → number•
cumuStd(v list number) → number•
cumuNorm(v list number) → list

4. Usage examples

```
import "std/c/tscumulative"
use tscumulative
```

```
create result = cumuSum(/* args */)
say result
```

```
create result = cumuMean(/* args */)
say result
```

```
create result = cumuMin(/* args */)
say result
```

```
create result = cumuMax(/* args */)
say result
```

```
create result = cumuVar(/* args */)
say result
```

```
create result = cumuStd(/* args */)
say result
```

5. Best practices

- Prefer ``import "std/c/tscumulative"`` at the top of your file.
- Run ``afrilang check`` before shipping.
- Combine with related stdlib modules from the same category.
- See `/stdlib/` for the live source and playground demos.
- Report issues on GitHub if an export is missing or undocumented.

6. Appendix — source

module tscumulative

```
function cumuSum(v list number) returns number
end
```

```
function cumuMean(v list number) returns number
end
```

```
function cumuMin(v list number) returns number
end
```

```
function cumuMax(v list number) returns number
end
```

```
function cumuVar(v list number) returns number
end
```

```
function cumuStd(v list number) returns number
end
```

```
function cumuNorm(v list number) returns list number
end
end
```

Référence rapide / Quick reference

- Import : `import "std/c/tscumulative"`
- Vérification : `afrilang check`
- Documentation : <https://afrilang.dev/stdlib/std/c/tscumulative/>

Source (excerpt)

```
module tscumulative

function cumuSum(v list number) returns number
end

function cumuMean(v list number) returns number
end

function cumuMin(v list number) returns number
end

function cumuMax(v list number) returns number
end

function cumuVar(v list number) returns number
end

function cumuStd(v list number) returns number
end

function cumuNorm(v list number) returns list number
end
end
```